

I programmi di prova

Sei programmi C# e un analizzatore in Python attorno alle due librerie: a che serve ciascuno, e quando si apre quello e non un altro

Perché ce ne sono così tanti

Su quasi tredicimila righe scritte, **le librerie vere sono il quaranta per cento**. Tutto il resto è l'infrastruttura costruita per dimostrare che quel quaranta per cento fa la cosa giusta, contro una macchina che nessuno aveva in ufficio.

Non è sovrabbondanza: ogni programma esiste perché a un certo punto è servito e non c'era. Il proxy è nato il giorno in cui è emerso che due comandi ravvicinati si perdevano e il Dev Kit non salvava il traffico. Il riempitore è nato quando il simulatore ha finito le monete a metà collaudo. L'analizzatore dei log è nato quando le sessioni sono diventate troppo lunghe da rileggere a mano.

Il modo più rapido di orientarsi è la domanda che ci si sta facendo:

La domanda che ti fai	Il programma
«che succede se gli mando questo?»	Banco di prova (WinForms o Compose)
«va ancora tutto?»	Collaudo
«ho scritto giusto la stringa del comando?»	Test offline
«che cosa è passato davvero sul filo?»	Proxy
«perché fallisce metà del collaudo?»	Riempitore (di solito è il simulatore vuoto)
«com'è andata la sessione di ieri?»	Analizzatore dei log
«come si usa questa libreria?»	Demo

1. Il banco di prova

C#: WinForms, 1.247 righe · Kotlin: Compose for Desktop, 1.247 righe

È la plancia manuale: una finestra con nove schede da cui si preme un bottone, si vede partire il comando e tornare la risposta, senza scrivere una riga di codice.

Quando si apre questo. Quando la domanda è «che cosa succede se gli mando questo?». È lo strumento dell'esplorazione — e della telefonata con il fornitore: si tiene aperto mentre si discute, si prova il comando contestato e si legge la risposta ad alta voce. Per la verifica ripetibile serve il collaudo, non questo.

Le nove schede sono le stesse nei due linguaggi, con gli stessi comandi: Connessione, Incasso, Contanti, Ricariche, POS, Display, Stampa, Sistema, Console. In basso, sempre visibile, il pannello del traffico colorato — invio in blu, ricezione in nero, esiti in verde, errori in rosso — e una finestra di log separata con filtri, ricerca e autoscorrimento.

Sopra il traffico ci sono due caselle che contano: **registra su file** e **diagnostica libreria**. La seconda accende il racconto interno — le pause imposte, i byte letti, le attese scadute. È quella da attivare prima di segnalare un difetto: dal solo traffico non si distingue un comando trattenuto dal client da un comando ignorato dalla macchina.

Una cosa da sapere prima di usarlo su una macchina vera. I bottoni di erogazione non hanno nessuna conferma: **il denaro esce al primo clic**. L'unica protezione è che il bottone si disabilita per la durata dell'operazione, il che evita il doppio clic ma non il primo. E la password di erogazione, che serve otto comandi diversi, sta in chiaro in una casella normale.

I due banchi sono equivalenti? Sì, comando per comando: non c'è un solo comando presente da una parte e assente dall'altra. Le differenze sono di ergonomia, e una sola conta davvero: sul banco Kotlin l'interrogazione dei movimenti è **fissa a oggi**, mentre su WinForms si sceglie l'intervallo di date. Le altre sono minori — il Compose non copia negli appunti ma esporta su file, e i campi numerici sono liberi invece che con limiti.

E scrivono le stesse righe di log. Dal 16 settembre anche gli importi: il banco Compose scriveva `incassato=800.0` dove il WinForms scrive `incassato=800,00`, e due log della stessa prova non si potevano confrontare. Ora tutti e due usano due decimali e il separatore della lingua del PC. Le cinque prove a mano del 16 settembre, eseguite sul simulatore in tutti e due i banchi, hanno dato righe identiche: `md/verbali/prove-banchi-ramo1-2026-09-16.md`.

2. Il collaudo

699 righe in C#, gemello in Kotlin

Esegue in sequenza tutta la libreria contro una macchina vera o contro il simulatore, e alla fine stampa una tabella di esiti riga per riga. Restituisce zero se non fallisce niente, quindi si può automatizzare.

Quando si apre questo. Dopo una modifica alla libreria, dopo un aggiornamento di firmware, o prima di consegnare. È il «*va tutto?*» di fine giornata — con una riserva: il collaudo passava anche con i tre difetti emersi dalla risposta di PayPrint (D1-D3 in `md/memoria-claude/esito-risposta-payprint.md`). Dall'11 settembre D1 e D3 sono corretti e il passo `[IN]+[CM]` manda CM a incasso aperto e controlla l'esito: se la libreria restituisse l'accettazione, il passo fallirebbe.

I gruppi e i passi

Il collaudo è diviso in gruppi indicabili come argomento, per una ragione pratica: quando un'area si rompe la si rilancia da sola in dieci secondi invece di rifare tutto.

Gruppo	Passi	Che cosa esercita
base	4	stato e decodifica giacenze, pulizia display, rinvio ultimo messaggio
cash	6	soglie, abilitazione tagli, fondo cassa, azzeramento cassetto di scarico
collect	4	incasso con parziali, incasso annullato, incasso chiuso trattenendo con CM a incasso aperto (fallisce se torna l'accettazione invece dell'esito), erogazione
erogazione	7	banconote, monete, spostamenti, azzeramento, incasso con timeout, incasso automatico
ricariche	7	le sette sessioni di ricarica, ognuna aperta e chiusa
display	4	testo, finestra a tre bottoni, lettura codice, lista
display2	2	testo in basso, finestra di input con tastiera
print	2	stato stampante e scontrino completo
print2	14	tredici comandi di stampa singoli più l'annullo di una stampa inesistente
system	5	totali POS, ultima transazione, incasso POS, ricarica, comando inesistente
pos2	3	ristampa, chiusura giornaliera, ricarica certificati del terminale carte
immagini	4	logo, immagine temporanea, rimozione, e il confronto fra i due incapsulamenti
movimenti	2	elenco di oggi e interrogazione per identificativo
Totale di default	64	
di	5	sonda cinque varianti del comando non documentato
riavvii	2	riavvio POS e riavvio macchina — esclusi di default

Attenzione al conteggio. Nel sorgente ci sono quarantanove chiamate, ma tre gruppi girano in ciclo: i passi realmente eseguiti sono **64** con i gruppi di default, **71** accendendo anche la sonda e i riavvii. Le ricariche eseguono sette sessioni da un solo punto di chiamata, `print2` tredici comandi più uno, `di` cinque varianti.

Il gruppo `di` merita una nota: quel comando **non è nei manuali**. Il collaudo prova cinque formati e riporta quale viene accettato, e la prima variante è stata estratta dal binario del Dev Kit di PayPrint. È esplorazione documentata, ed è il motivo per cui quel gruppo sta fuori dai predefiniti.

Sette profili di avvio già pronti: collaudo completo sul simulatore, solo base, solo incassi ed erogazioni, solo display e stampa, la sonda, e i due equivalenti verso la macchina reale. Il gruppo dei riavvii non è raggiungibile da nessun profilo — va digitato a mano, ed è voluto.

3. Il proxy

459 righe

Si mette in mezzo fra un qualsiasi client e la macchina e registra tutto quello che passa nei due sensi. È un proxy TCP trasparente: ascolta su una porta, apre una seconda connessione verso la destinazione, e avvia due pompe in parallelo che leggono, registrano e riscrivono identico.

Quando si apre questo. Quando il colpevole non è chiaro. Il banco dice cosa hai chiesto tu; il proxy dice cosa è **realmente passato sul filo**, anche se a parlare è il software del fornitore.

Perché non è un semplice «vedi i byte»

Registra ogni segmento TCP **così com'è arrivato**, non il messaggio già ricomposto. È l'unico modo per vedere due comandi accorpati e per misurare la pausa che un client tiene fra un comando e l'altro. E su quella base fa due analisi mirate:

1. **Comandi accorpati.** Prova a spezzare il segmento usando una tabella di trenta comandi a lunghezza nota. Se ne trova più di uno stampa in rosso: *«N comandi in un unico segmento. Il pagAmico esegue solo il primo»*. Sui comandi a lunghezza variabile si ferma senza indovinare. Il messaggio, scritto così nel codice, vale per quello che si è osservato: il simulatore, con comandi senza terminatore. Per il fornitore basta che ogni comando termini con CR o CR+LF (domande 1.1 e 1.2, vedi `md/memoria-claude/esito-risposta-payprint.md`): a quel punto anche due comandi nello stesso segmento dovrebbero passare. Sul simulatore è provato dal 14 settembre, e passa anche senza terminatore (`md/verbalì/prova-terminatore-cr-2026-09-14.md`); sulla macchina va ancora provato. Il CR è il default delle librerie, e nei banchi la casella del terminatore parte con `\r`. Con il CR l'avviso rosso compare lo stesso — il proxy toglie CR e LF solo agli estremi del segmento — e fa fede solo la risposta al secondo comando.
2. **Comandi troppo ravvicinati.** Sotto i trenta millisecondi segnala il rischio anche se i comandi erano in due segmenti distinti. Quella soglia è misurata, non stimata, ma sul simulatore e senza terminatore: è la stessa che regge il default a ottanta millisecondi della libreria. Per il fornitore, sulla macchina e con il CR o CR+LF, la pausa non serve; gli ottanta millisecondi restano il default finché la raffica con CR non regge su una macchina vera (esito, sezione 3, punto 11).

In più abbina risposte a comandi, e segnala il caso più interessante di tutti: **«risposta non sollecitata: il client non aveva comandi in sospeso»**. Può essere la firma di un messaggio spontaneo — cioè la cosa che si vuole scoprire: su barcode, BT1 ed EX il fornitore non ha risposto. Ma il proxy abbina un solo segmento a ogni comando, quindi compaiono così anche i parziali e l'esito di un incasso e il secondo frame dopo un CM (D1). Dall'11 settembre anche le librerie hanno un segnale simile, l'evento dei **frame orfani**: un messaggio che nessuna attesa riconosce.

A fine sessione stampa un riepilogo con le statistiche che contano: pausa fra comandi del client minima, media e massima, e tempo di risposta della macchina.

L'uso più prezioso, ed è il motivo per cui esiste. Si punta il **Dev Kit di PayPrint** sul proxy invece che sul simulatore: lui crede di parlare con la macchina, e intanto ogni byte finisce nel log. Quello che manda il loro programma resta la risposta autorevole sul formato dei comandi; su terminatore e pause, che erano nella stessa lista, ha già risposto il fornitore (domande 1.1 e 1.2: CR o CR+LF, nessuna pausa), e il traffico del Dev Kit è un modo di verificarlo sul filo.

Una cosa non ovvia: ascolta su tutte le interfacce, non solo su locale. Si può quindi mettere il proxy su un PC e far passare da lì il traffico di un'altra macchina della rete, senza toccare né il client né il pagAmico.

4. Il riempitore del simulatore

248 righe

Mette contanti dentro al simulatore. Non con un comando di ricarica — che sul simulatore non aggiunge nulla, perché lì il contante lo mette l'operatore fisico — ma **eseguendo incassi finti ripetuti** finché le giacenze arrivano all'obiettivo, alternando i tagli così che la macchina si ritrovi spiccioli di ogni valore e riesca a comporre qualsiasi resto.

Quando si apre questo. Prima di tutto il resto, una volta sola, quando il simulatore è vuoto. Un simulatore a zero non può dare resto, quindi metà dei passi del collaudo fallirebbe per un motivo che non c'entra con il codice.

Non va usato su una macchina reale: esegue incassi veri. Nessuno dei quattro profili di avvio punta a un indirizzo diverso da 127.0.0.1, ed è voluto.

La cosa non ovvia è il rilevatore di giro a vuoto. Se nel simulatore non è attivo l'interruttore *«il cliente inserisce l'importo esatto»*, il cliente virtuale arrotonda per eccesso: cinque giri su sei guadagnano e il sesto restituisce come resto tutto quello che i primi cinque avevano messo dentro. Il saldo resta a zero mentre i singoli giri sembrano rendere. Il programma confronta quindi su una finestra lunga quanto la scala dei tagli, e se dopo quella finestra la giacenza non è salita si ferma e **dice esattamente quale interruttore attivare**. Senza quel controllo si macinerebbe fino a sessanta giri senza accorgersi di girare a vuoto.

È una scoperta che è costata mezza giornata, ed è finita fra i suggerimenti a PayPrint per la loro versione BETA.

Non serve per la prova del resto insufficiente. Vale la pena dirlo perché è un errore facile: il riempitore **riempie soltanto**, si ferma appena la giacenza raggiunge l'obiettivo. Per svuotare la cassa monete — che serve per provare una restituzione incompleta — si usano i comandi di manutenzione dal banco di prova.

5. I test offline

289 righe

Verificano che le stringhe di comando prodotte dalla libreria siano identiche, carattere per carattere, agli esempi stampati nei manuali, e che il decodificatore legga correttamente il JSON documentato. Nessuna macchina, nessuna rete: girano in un secondo.

Quando si apre questo. Dopo ogni modifica alla libreria, prima di accendere qualsiasi cosa. È il primo controllo della giornata: se fallisce qui, non ha senso collegarsi.

Non usano nessun framework di test: sono un normale programma con tre primitive scritte a mano. Scelta coerente col resto — zero dipendenze, si lancia e basta.

Che cosa verificano i test (171 in C#, 172 in Kotlin)

Dove i numeri differiscono, il primo è C# e il secondo Kotlin. L'unico test in più di Kotlin riguarda la cancellazione delle coroutine: in C# l'annullo col token restituisce l'esito, in Kotlin la cancellazione deve propagarsi subito e l'esito viaggia con l'eccezione.

Sezione	Asserzioni	Che cosa verifica
Comandi	28	le stringhe di ventuno comandi distinti contro gli esempi letterali del manuale, più tre casi che devono fallire (importo troppo lungo, password troppo lunga, soglia minima maggiore della massima)
Framer	12	il pezzo più delicato: JSON spezzato su due segmenti, due JSON nello stesso segmento, il marcatore delle liste, graffe e virgolette dentro lo scontrino POS che non devono chiudere il messaggio
Risposta JSON	21	la mappatura del JSON di esempio, le giacenze per taglio, e le incoerenze del firmware : un nome di campo con un refuso, uno abbreviato, uno con uno spazio iniziale
Stampa	16	undici comandi contro il manuale, la struttura di uno scontrino, la decodifica dello stato stampante
Display	9	i comandi contro gli esempi del manuale, il JSON compatto delle liste e il limite di caratteri
Sequenze di incasso	52 / 53	dall'11 settembre: il client vero contro un finto pagAmico su 127.0.0.1. OK p BUSY p IN, OK CMD ERROR IN, BUSY e ER E100/99 prima dell'OK, CM/OK seguito dal CM finale, CM/NO, CM prima dell'OK, una sola chiusura per incasso, annullo del chiamante prima e dopo l'OK, invii bloccati a incasso aperto, frame orfani, PO/IM/I2, chiusura forzata dal pannello, timeout, caduta e Disconnect() a incasso aperto, caduta mentre il CM aspetta l'OK; solo Kotlin: l'esito AN arriva anche con l'eccezione di cancellazione del chiamante
Comandi semplici e invio	11	un CM in ritardo non fa da risposta a ST; OK, testo di errore e LO; la pausa di 80 ms e il CR con la configurazione di default; il terminatore CR+LF; i due incapsulamenti delle immagini, byte per byte; il keepalive TCP regolato alla connessione
Registro su file	8	due scrittori sullo stesso file, tre processi veri sullo stesso file, cambio di giorno, due logger avviati in giorni diversi, ripulitura dei caratteri di controllo, troncamento, logger spento
Errori e display	13	il comando DI, le descrizioni dei codici, la stringa di stato, il riconoscimento di BUSY

Che cosa NON coprono

È la parte importante da sapere prima di fidarsi del verde.

- **Una parte del client.** I test aprono un socket verso un finto pagAmico e coprono incasso, commit, comandi semplici, pausa e terminatore. Restano non verificati la riconnessione, i timeout dei comandi fuori dall'incasso, e **la raffica**: la pausa è provata, ma che la macchina regga due comandi chiusi da CR senza pausa si vede solo su una macchina vera. E il finto pagAmico risponde con le sequenze che conosciamo dal simulatore e dal manuale: quelle della macchina vera restano da vedere.
- **Il keepalive con un cavo staccato.** I test verificano i valori applicati al socket, non dopo quanto una caduta vera viene vista: è la prova 10 di `md/checklist/checklist-macchina-reale.md`.
- **Quali incapsulamenti delle immagini accetta la macchina:** i test verificano i byte che partono, non chi li capisce. Poi quasi tutti i codici di errore del manuale, e i programmi di prova stessi.

In sintesi. Gli 86 test sui manuali coprono bene **la traduzione fra i manuali e le stringhe** — comandi in uscita, JSON in entrata — e il framing, che sono le due cose in cui è facile sbagliare in silenzio. I 63 (64 in Kotlin) contro il finto pagAmico coprono lo **stato dell'incasso**, dove stavano D1 e D3, e i comandi semplici; i 21 sul registro e sugli errori il resto della libreria. Quello che resta al collaudo, e alla macchina vera, è il comportamento del dispositivo.

6. La demo

136 righe

Il «guarda come si fa»: stato macchina, un incasso, elenco movimenti di oggi. Tre metodi, cento righe, tutto quello che serve per capire l'uso normale.

Quando si apre questo. All'inizio, o quando bisogna spiegare l'integrazione a qualcun altro. È il file da mandare a chi deve integrare il pagAmico nel proprio gestionale: è scritto per essere copiato. Prima di mandarlo, però, due avvertenze: aspetta l'incasso col timeout di default di cinque minuti e allo scadere manda un AN, mentre il fornitore dice di non mettere timeout (domanda 2.4 in md/memoria-claude/esito-risposta-payprint.md). L'attesa che usa, fino al 10 settembre, la chiudeva un testo qualsiasi (difetto D3, corretto).

La cosa non ovvia è didattica: durante l'incasso lancia un'attività in secondo piano che aspetta un tasto e annulla. Il punto è che annullare **non abbandona l'attesa**: la libreria manda il comando di annullo alla macchina e continua ad aspettare l'esito reale, l'AN. Se l'AN porti anche l'importo restituito non è ancora provato: nei log non ce n'è uno con denaro dentro (domanda 2.2 in md/memoria-claude/esito-risposta-payprint.md). È la differenza fra «smetto di ascoltare» e «annullo davvero», ed è il comportamento giusto quando c'è di mezzo del denaro.

7. L'analizzatore dei log

466 righe, in Python

Non parla con niente: legge file già scritti su disco. Si può lanciare a macchina spenta, a simulatore spento, il giorno dopo. Ed è **l'unico strumento che serve tutte e due le librerie**, perché i due registri producono lo stesso formato.

Quando si lancia. A fine giornata di prove, per sapere com'è andata senza rileggere migliaia di righe. Dopo che qualcosa è andato storto, per capire se la colpa è del trasporto o della macchina. Prima di aprire una segnalazione a PayPrint. E per sapere che cosa manca ancora da provare.

Produce un rapporto per file, con sei controlli:

1. **Comandi rimasti senza risposta** — con due filtri che evitano i falsi allarmi: gli otto comandi che per protocollo non rispondono mai sono esclusi, e quattro che rispondono ma non subito finiscono in una riga a parte.
2. **Comandi inviati troppo ravvicinati** — fino alla risposta di PayPrint il difetto numero uno del progetto, trovato a posteriori nel log senza dover rifare la prova. Per il fornitore, sulla macchina e con il CR o CR+LF, la pausa non serve: il controllo conta per le sessioni senza terminatore (tutte quelle fino al 14 settembre) e, con il CR ormai di default, finché la raffica non regge sulla macchina: sul simulatore regge.
3. **Risposte di errore**, col codice tradotto in italiano.
4. **Stato delle scorte**, decodificando la stringa di stato cifra per cifra: è il campo che dice se la macchina sta per non riuscire più a dare il resto.
5. **Attese scadute e messaggi non sollecitati** — funziona solo se la diagnostica era accesa.
6. **Copertura**: quali comandi sono stati provati e soprattutto **quali non sono mai stati toccati** in quella sessione.

La parte più utile è il verdetto finale, che distingue due cose che si confondono facilmente: **una risposta di errore non è un guasto della comunicazione**. Se la macchina ha risposto con un errore, ha ricevuto e capito il comando.

Un limite da conoscere: l'analizzatore **scarta di proposito** i tracciati del proxy, che hanno un formato proprio e il riepilogo già in fondo al file. Il tracciato del proxy si legge direttamente; l'analizzatore serve per il registro *di libreria* della stessa sessione.

8. Come si lanciano

Dalle IDE, non da terminale. È una regola, non una preferenza: ogni programma che vuole argomenti ha un profilo salvato nel repository, numerato nell'ordine in cui si esegue. Il menu a tendina dell'IDE è la sequenza delle prove.

	Visual Studio	IntelliJ IDEA
test offline	progetto Tests	configurazione 1
accendere il simulatore	Dev Kit, sezione Simulatore	—
riempire le giacenze	progetto Fill, profilo 2	—
collaudo completo	progetto LiveTest, profilo 1	configurazione 2
prove a mano	progetto WinForms	configurazione 3
guardare il traffico	progetto Tap, profilo 1	—

La ragione è documentata da un incidente vero: in un log del 3 settembre compare `logger` agganciato a `--nologo:9100` — un avvio da riga di comando in cui l'opzione è stata scambiata per il nome dell'host. Undici secondi dopo la riga è corretta. I profili numerati eliminano quella classe di errori.

Nota sull'indirizzo della macchina reale. In tutti i profili compare un indirizzo che è un **segnaposto**, non una macchina esistente. Il giorno in cui arriva il pagAmico si legge l'indirizzo vero sul display, in *Menu Servizi*, e si corregge una volta sola nei profili.

Il dettaglio operativo — passo per passo, con che cosa deve succedere a ogni schermata — sta nella *Guida alle prove*.